

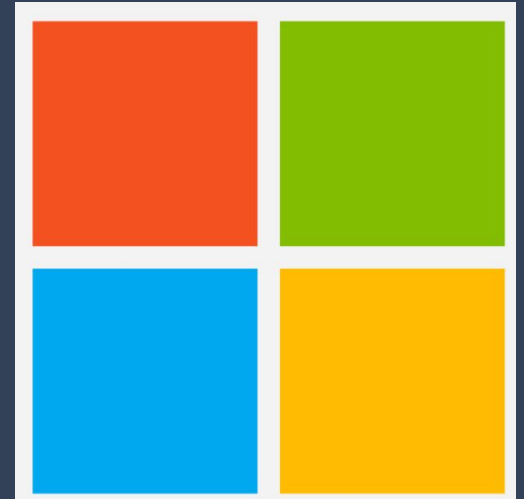
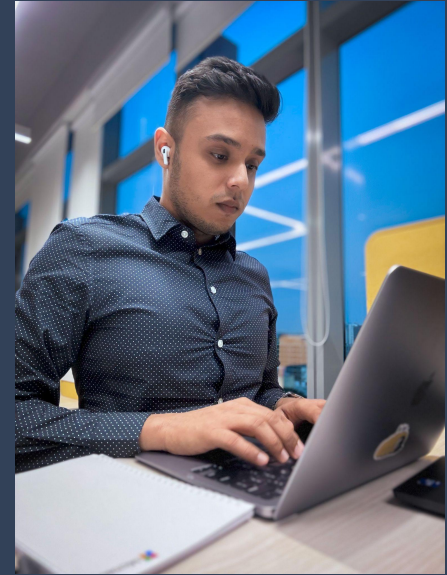
Python for GenAI



Instructor: Anshaj Khare

Introduction – Anshaj Khare

- ✓ I have been working on data science since **2017**
- ✓ I've worked for 4 startups before joining Microsoft
- ✓ I have been working at Microsoft for about 4 years as an ML Engineer
- ✓ In my career, I've worked as a backend engineer, a devops engineer, a data engineer and as a Machine learning engineer
- ✓ Have taught over 4000+ learners - Python, Data Science and Gen AI
- ✓ Author of a blog on everything related to computation: <http://compute.anshaj.dev/>
- ✓ Github: <http://github.com/anshajk>



Frequently Asked Question - 1

I don't have to code in my job role, why am I learning Python here? How will it help me?

Don't worry, you don't have to learn Python to the extent of a Python Developer. You only need to be able to comprehend the code that would be used in the upcoming modules in the course.

In GenAI, Python is just a tool! With this Crash Course, you would understand just enough on how to operate the tool and that's all you need!

Frequently Asked Question - 2

I have never coded in Python before. Will I become an expert in Python after completing this module?

This is a Python Crash Course - tailored specifically to equip you with enough knowledge to help you navigate through the Applied GenAI Course smoothly.

Generative AI technologies are implemented through Python, and you will encounter multiple coding demos in Python throughout the course. This Python Crash Course includes just enough details so that the coding demos in the further modules are comprehensible.

If you have never coded in Python before, you may need to refer to external resources too. If you need help with the resources, please contact studentsupport@interviewkickstart.com

If you are already familiar with Python programming, this module may seem like a basic refresher for you - and that's exactly what it's meant to be!

Today's Agenda



Break

Break

Break

1



Intro to
Python,
Data Types
& Strings

2



Basic Data
Structures
in Python

3



Conditionals,
Functions in
Python

4



Pandas
Basics

Introduction to Python

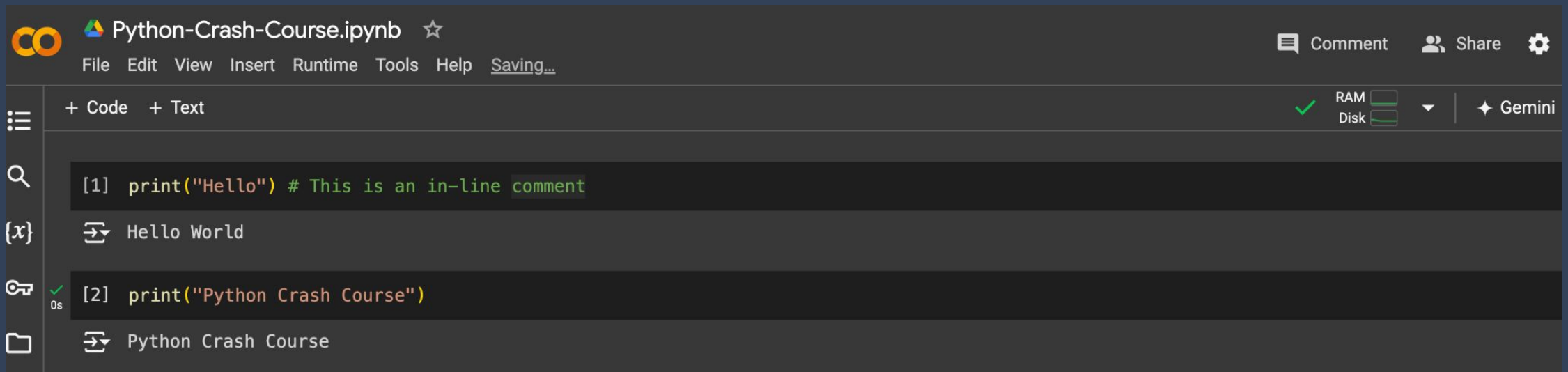
Introduction

- Python is a versatile and powerful programming language used in various domains. From web development to data science, Python has become a language of choice for many.
- Python is a **high-level**, **interpreted**, and object-oriented programming language.
- Python was created by Guido Van Rossum and released in 1991.
- There are several Integrated Development Environments (**IDEs**) available for coding in Python, including PyCharm, Visual Studio Code, Spyder, and **Jupyter**



Why Google Colab?

- It allows you to write your code in separate blocks or cells, and you can execute these cells one at a time.



Role of Python in Generative AI

- **Primary Language for AI:** Python's simplicity and readability make it the go-to language for AI development, supported by libraries like TensorFlow, PyTorch, and Keras.
- **Generative Models:** Python is used to implement various generative models such as GANs, VAEs, and Transformers for applications like image, text, and audio generation.
- **Integration and Deployment:** Seamless integration with APIs, web services, and cloud platforms (e.g., AWS, Google Cloud) for scalable deployment of AI models.
- **Extensive Libraries:** Essential libraries like NumPy and Pandas facilitate data manipulation and preprocessing tasks critical for AI development.
- **Vibrant Community:** A large, active community and abundant educational resources support ongoing learning and development in Python for generative AI.

Variables and Data Types

Basic Python Programming - Variables and Data Types

- **Variables:**

- Store values for later use
- No need to declare data type in advance

- **Data types:**

- Common types: int, float, str, bool, list, tuple, set and dictionary
- Automatically assigned based on the value

- **Integer:** Example: `a = 10`

- **Float:** Example: `b = 6.2`

String: Example: `c = "Matt"`

Boolean: Example: `d = True`

Operators



```
x = 10
y = 3
# Arithmetic operators
print("x + y =", x + y)
print("x - y =", x - y)
# Comparison operators
print("x == y?", x == y)
print("x < y?", x < y)
```



```
X = 10
```

```
y = 20
```

```
name = "Alice"
```

```
# Logical operators
```

```
print("x > 0 and y > 0:", x > 0 and y > 0)
```

```
print("not x < 0:", not x < 0)
```

```
# Other operators
```

```
print("'A' in name:", 'A' in name)
```

```
print("x is y:", x is y)
```

Strings

Python String - Defining and Using in Python

- Ordered, immutable sequence of characters
- Enclosed in single or double quotes (" or """)



```
# Using single quotes for a simple string
single_quoted_string = 'Hello, World!'

# Using double quotes for a simple string
double_quoted_string = "Hello, World!"

# Using double quotes to avoid escape character
double_without_escape = "It's a beautiful day."
single_with_escape = 'It\'s a beautiful day.'

# Multi-line string with triple quotes
multi_line_string = """
This is a multi-line string.
It can span multiple lines without needing any escape characters.
"""
```


Important String Methods



Converts all characters in the string to lowercase.

`str.lower():`

Example: `"Hello".lower()` returns `"hello"`.

Converts all characters in the string to uppercase.

`str.upper():`

Example: `"hello".upper()` returns `"HELLO"`.

Capitalizes the first character of the string and lowercases the rest.

`str.capitalize():`

Example: `"hello world".capitalize()` returns `"Hello world"`.

Converts the first character of each word to uppercase and the rest to lowercase.

`str.title():`

Example: `"hello world".title()` returns `"Hello World"`.

Removes any leading and trailing whitespace

`str.strip():`

Example: `" hello ".strip()` returns `"hello"`.

Removes any leading whitespace

`str.lstrip():`

Removes **any** leading whitespace.

Example: `" hello ".lstrip()` returns `"hello "`.



Important String Methods



Removes any trailing whitespace

`str.rstrip():`

Example: `" hello ".rstrip()` returns `"hello"`.

Splits the string into a list of substrings based on a specified delimiter

`str.split():`

Example: `"hello world".split()` returns `["hello", "world"]`.

Joins a list of strings into a single string with a specified delimiter

`str.join():`

Example: `" ".join(["hello", "world"])` returns `"hello world"`.



Advanced String Operations in Python

- String Concatenation:

- ```
String Concatenation into one string.
str1 = "Hello"
str3 = "World"
concatenated_string = str1 + " " + str3
print(concatenated_string) # Output: Hello World
```

- String Formatting

- ```
name = "Alice"  
age = 30  
formatted_string = f"My name is {name} and I am {age} years old."  
print(formatted_string) # Output: My name is Alice and I am 30 years  
old.
```

- ```
formatted_string = "My name is {} and I am {} years old.".format(name, age)
print(formatted_string) # Output: My name is Alice and I am 30 years old.
```

# Advanced String Operations in Python



```
text = """
Python is a versatile and powerful programming language used in various domains. From web development to
data science, Python has become a language of choice for many.
Python is a high-level, interpreted, and object-oriented programming language.
Python was created by Guido Van Rossum and released in 1991.
Python's readability and simplicity make it an ideal language for beginners.
It's widely used in industry, making it a valuable skill for future opportunities.
"""

Define the prompt for the AI model, asking it to summarize the provided text
prompt = f"""
Summarize the text delimited by triple backticks into a single sentence.
```{text}```
"""

# Generate the summary response by calling LLM
response = call_LLM(prompt)

# Print the generated summary
print(response)
```

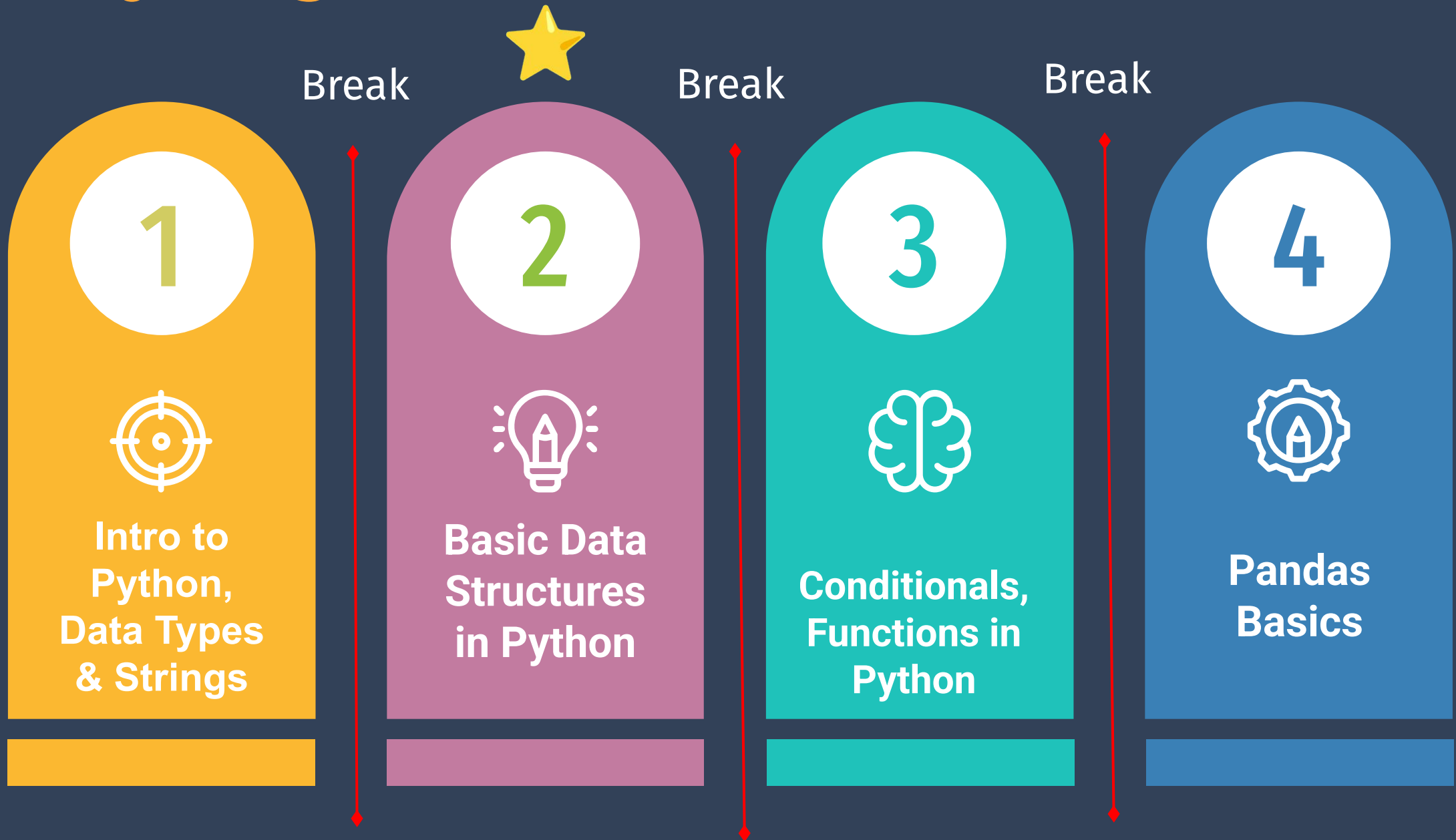


Break

Time!



Today's Agenda



Python Data Structures

Lists

Python List - Overview and Basic Operations

- **Definition:**

- Ordered, mutable collection of items
- Can contain elements of different data types
- Created using square brackets []

- **Usage:**

- Create variable-length and mutable sequences of objects
- Store objects of any type
- Store multiple types of objects together



Characteristics of a Python List

The various characteristics of a list are -

- **Ordered:** Lists maintain the order in which the data is inserted.

```
fruits = ["apple", "banana", "cherry"]  
print(fruits) # Output: ['apple', 'banana', 'cherry']
```

- Accessing Items using their index (Zero-based, Negative indexing).

```
first_item = fruits[1]  
print(first_item) # Output: 'banana'
```

- **Dynamic:** List can expand or shrink automatically to accommodate the items accordingly.
- **Duplicate Elements:** Lists allow us to store duplicate data

- **Heterogenous:** Lists can store elements of various data types.

```
mixed_list = ["apple", 42, 3.14, [1, 2, 3]]  
print(mixed_list) # Output: ['apple', 42, 3.14, [1, 2, 3]]
```

- **Mutable:** In list element(s) are changeable. It means that we can modify the items stored within the list.

```
# Adding an item  
fruits.append("date")  
print(fruits) # Output: ['apple', 'banana', 'cherry', 'date']
```

```
# Removing an item  
fruits.remove("banana")  
print(fruits) # Output: ['apple', 'cherry', 'date']
```

```
# Changing an item  
fruits[1] = "blueberry"  
print(fruits) # Output: ['apple', 'blueberry', 'date']
```

Slicing in Python

Slicing is a technique in Python used to extract a subset of elements from a sequence (such as a list, tuple, or string) by specifying a range of indices.

`sequence[start:stop:step]`

start: The starting index of the slice (inclusive). Defaults to 0 if not specified.

stop: The ending index of the slice (exclusive). The slice includes elements up to, but not including, this index.

step: The step size, or the interval between indices in the slice. Defaults to 1 if not specified.

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
# Slicing from index 2 to 5
```

```
subset = numbers[2:6]  
print(subset) # Output: [2, 3, 4, 5]
```

```
# Slicing with step size of 2
```

```
subset = numbers[1:8:2]  
print(subset) # Output: [1, 3, 5, 7]
```

```
# Define a string
```

```
text = "Hello, World!"
```

```
# Slicing from index 7 to the end
```

```
substring = text[7:]  
print(substring) # Output: "World!"
```

```
# Slicing from the beginning to index 5
```

```
substring = text[:5]  
print(substring) # Output: "Hello"
```

Do you wonder how negative indexing works in Python Slicing?



Python Slicing - Negative Indexing

- Negative indices:
 - Can be used for slicing to refer to elements from the end of the sequence



```
# Create a list of numbers
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Using negative index to slice the list from the second-
# to-last element to the fifth-to-last element (exclusive)
slice_negative = numbers[-5:-2]
print("Negative index slice:", slice_negative) # Output:
[6, 7, 8]
```

Tuples

Python Tuple - Overview & Characteristics

The various characteristics of a Tuple are -

- **Ordered Collection**

```
fruits = ("apple", "banana", "cherry")  
print(fruits) # Output: ('apple', 'banana', 'cherry')
```

- Accessing Items using their index (zero-based, negative).

```
first_item = fruits[0]  
print(first_item) # Output: 'apple'
```

- **Immutable Collection**: We can not add, remove or change an item in Tuples.

- **Different Data Types**

```
mixed_tuple = ("apple", 42, 3.14, (1, 2, 3))  
print(mixed_tuple) # Output: ('apple', 42, 3.14, (1, 2, 3))
```

Set

Characteristics of a Python Set

The various characteristics of a set are -

- **Unordered Collection:**

```
fruits_1 = {"apple", "banana", "cherry"}  
fruits_2 = {"apple", "cherry", "banana"}
```

- **Unique Items::**

```
fruits = {"apple", "banana", "cherry", "apple"}  
print(fruits) # Output: {'apple', 'banana', 'cherry'} (duplicates are removed)
```

- **Mutable Collection:**

```
# Adding an item  
fruits.add("date")  
print(fruits) # Output: {'apple', 'banana', 'cherry', 'date'}
```

```
# Removing an item  
fruits.remove("banana")  
print(fruits) # Output: {'apple', 'cherry', 'date'}
```

Characteristics of a Python Set

Different Data Types (Sets cannot contain mutable elements like lists or dictionaries)

```
mixed_set = {"apple", 42, 3.14, (1, 2, 3)}  
print(mixed_set)  # Output: {42, 3.14, 'apple', (1, 2, 3)}
```

Dictionary in Python

Characteristics of Dictionaries in Python

Key-Value Pairs

```
student_ages = {"Alice": 25, "Bob": 22, "Charlie": 23}  
print(student_ages) # Output: {'Alice': 25, 'Bob': 22, 'Charlie': 23}
```

Unique

- If a dictionary contains duplicate keys, the last key-value pair will be used.

```
student_ages = {"Alice": 25, "Bob": 22, "Charlie": 23, "Alice": 26}  
print(student_ages) # Output: {'Alice': 26, 'Bob': 22, 'Charlie': 23}
```

Accessing Values::

```
age_of_alice = student_ages["Alice"]  
print(age_of_alice) # Output: 26
```

Characteristics of Dictionaries in Python

Adding and Modifying Items

```
# Adding a new key-value pair
student_ages["David"] = 24
print(student_ages) # Output: {'Alice': 26, 'Bob': 22, 'Charlie': 23, 'David': 24}
```

```
# Modifying an existing key-value pair
student_ages["Alice"] = 27
print(student_ages) # Output: {'Alice': 27, 'Bob': 22, 'Charlie': 23, 'David': 24}
```

Different Data Types

- Keys must be immutable (e.g., strings, tuples).
- Values can be of any data type.

```
mixed_dict = {"name": "Alice", "age": 25, "is_student": True, "grades": [85, 90, 92]}
print(mixed_dict)
```

Practical Examples

1. **Storing Data with Different Attributes:** Dictionaries are ideal for storing data with different attributes, such as information about users, products, or any entities with multiple properties.

```
user = {  
    'name': 'John Doe',  
    'age': 30,  
    'email': 'john@example.com',  
    'city': 'New York'  
}
```

1. **Retrieving Information Efficiently:** Dictionaries offer fast lookups, allowing you to retrieve information quickly using keys.

```
# Accessing user's email  
email = user['email']
```


Practical Examples

3. **Mapping Unique IDs to Objects:** Dictionaries are useful for mapping unique identifiers to objects, facilitating efficient retrieval of objects based on their IDs.

```
# Mapping student IDs to their information
students = {
    1001: {'name': 'Alice', 'age': 20},
    1002: {'name': 'Bob', 'age': 22},
    # More students...
}
```

3. **Configuration Settings:** Dictionaries are commonly used to store configuration settings for applications.

```
# Application settings
settings = {
    'debug': True,
    'log_level': 'INFO',
    'timeout': 30
}
```



Quiz

What is the output of the following code snippet?

```
my_dict = {'a': 1, 'b': 2, 'c': 3}  
print(my_dict['d'])
```

A 0

B None

C IndexError: 'd'

D KeyError: 'd'

Quiz - Solution

What is the output of the following code snippet?

```
my_dict = {'a': 1, 'b': 2, 'c': 3}  
print(my_dict['d'])
```

A

0

B

None

C

IndexError: 'd'

D

KeyError: 'd'

Break

Time!



Today's Agenda

1



Intro to
Python,
Data Types
& Strings

Break

2



Basic Data
Structures in
Python

Break

3



Conditionals,
Functions in
Python

Break

4



Pandas
Basics



Conditionals in Python : If-else statements

Indentation in Python

Indentation in Python is a fundamental aspect of the language's syntax. Unlike many other programming languages that use braces `{}` to delimit blocks of code, Python uses indentation to define the scope and structure of loops, conditionals, functions, and other control structures.

Consistency is Crucial:

1. All lines of code in a block must be indented the same amount.
2. Python does not enforce a specific number of spaces, but the convention is to use 4 spaces per indentation level

Indentation Levels:

2. Each indentation level indicates a new block of code.
3. Indentation helps to visually distinguish blocks of code that are nested inside other blocks.

Indentation for Control Structures:

1. Indentation is used to define the scope of loops (for, while), conditionals (if, elif, else), functions (def), classes (class), and other constructs.

Conditional Statements in Python

- ✓ The **if statement** checks a condition, and if the condition is True, it executes the block of code that follows.
- ✓ The **elif statement** checks another condition if the previous if or elif condition was False. It can be used multiple times.
- ✓ The **else statement** is executed if none of the preceding if or elif conditions are True.

```
x = 10
if x > 5:
    print("x is greater than 5")
```

```
x = 10
if x > 15:
    print("x is greater than 15")
elif x > 5:
    print("x is greater than 5 but less than
or equal to 15")
```

```
x = 3
if x > 5:
    print("x is greater than 5")
elif x > 2:
    print("x is greater than 2 but less
than or equal to 5")
else:
    print("x is 2 or less")
```

Python If-Else Statements - Example



```
age = 18
```

```
if age >= 18:  
    print ("You are eligible to vote.")
```

```
else:  
    print ("You are not eligible to vote.")
```

Loops in Python

Python Loops - Types and Usage

In Python, loops are used to repeatedly execute a block of code as long as a certain condition is true or for a specified number of times.

1. **'for' Loops:** iterate over a sequence (such as a list, tuple, string, or range)
2. **'while' Loops:** used to repeatedly execute a block of code as long as a specified condition is true

Python Loops - Types and Usage

Two types of loops:

- **For loop:** Iterate over a sequence (e.g., list, tuple, string)
- **While loop:** Execute code block while a condition is true

Loops - For

For Loop

Used for iterating over a sequence (such as a list, tuple, dictionary, set, or string).

Syntax

```
for variable in sequence:  
    # code block to execute
```

Example

```
fruits = ["apple", "banana", "cherry"]  
for fruit in fruits:  
    print(fruit)  
# Output:  
# apple  
# banana  
# cherry
```

Loops - While

Repeatedly execute a block of code as long as a condition is true.

Syntax

```
while condition:  
    # code block to execute
```

Example

```
count = 1  
while count <= 3:  
    print(count)  
    count += 1
```

```
# Output:  
# 1  
# 2  
# 3
```

**Do you know what are loop
control statements?**



Python Loops - Loop Control

- **Loop control statements:**

- **break:** Exit the loop immediately
- **continue:** Skip the rest of the loop body and start the next iteration

Output of the previous code



Output:

0
1
2

Nested Loops in Python

- Nested loops:
 - Loops can be placed inside other loops to create complex looping structures



Python Loops - Nested Loops



```
# List of items
items = ['a', 'b', 'c', 'd']
# List to hold the pairs
pairs = []
# Nested loop to create pairs
for i in items:
    for j in items:
        # Append the pair to the pairs list
        pairs.append((i, j))
# Print the list of pairs
print(pairs)
```

Python Loops - Loop Control and Nested Loops

- Best practices:
 - Keep loop bodies simple and readable
 - Avoid excessive nesting



Functions

What are Functions?



??

Python Functions - Definition and Syntax

Functions:

- Reusable pieces of code that perform a specific task
- Improve code readability, modularity, and maintainability

Function call:

- Invoke a function by its name, followed by parentheses containing arguments
- Defined using the def keyword.
- Parameters are variables listed inside the parentheses in the function definition.
- They allow you to pass information into functions.
- The return statement is used to exit a function and return a value.


```
def function_name(arguments):  
    # function body  
  
    return
```

Here,

- `def` - keyword used to declare a function
- `function_name` - any name given to the function
- `arguments` - any value passed to function
- `return` (optional) - returns value from a function

Let's see an example,

```
def greet():  
    print('Hello World!')
```

Function

Syntax

```
def function_name(parameters):  
    # code block to execute  
    return result
```

```
texts = [  
    "Python is a versatile ...  
]
```

```
summaries = []  
for text in texts:  
    prompt = f"""  
        Summarize the following text into a  
single sentence:  
        {text}  
        """  
    summary = call_LLM(prompt)  
    summaries.append(summary)  
print(summaries)
```

```
texts = [  
    "Python is a versatile ...  
]
```

```
def summarize_texts(texts):  
    summaries = []  
    for text in texts:  
        prompt = f"""  
            Summarize the following text into a single  
sentence:  
            {text}  
            """  
        summary = call_LLM(prompt)  
        summaries.append(summary)  
    return summaries
```

```
Summary_results = summarize_texts(texts)  
print(Summary_results )
```

**What's the difference between
arguments and parameters?**



Python Functions - Parameters and Arguments

- **Parameters vs. arguments:**
 - **Parameters:** Variables defined in the function's signature
 - **Arguments:** Values passed to the function when it's called



Function Parameter and Argument

✓ Positional arguments:

Positional arguments are the most common type. They are passed to the function in the same order as the parameters are defined

```
def subtract(a, b):  
    return a - b
```

```
result = subtract(10, 5)  
print(result) # Output: 5
```

✓ Keyword arguments:

Keyword arguments are passed to the function by explicitly specifying the parameter name.

```
def greet(name, message):  
    print(f"{message}, {name}!")
```

```
greet(name="Alice", message="Hello")  
greet(message="Hi", name="Bob")
```

✓ Default parameters:

Default parameters allow you to specify default values for parameters. If an argument is not provided, the default value is used.

```
def greet(name, message="Hello"):  
    print(f"{message}, {name}!")
```

```
greet("Alice")  
greet("Bob", "Hi")
```

Function Parameter and Argument

✓ Variable-length arguments:

i. ***args:**

Allows a function to accept any number of positional arguments and it collects all positional arguments into a tuple

```
def multiply(*args):  
    result = 1  
    for num in args:  
        result *= num  
    return result
```

```
print(multiply(1, 2, 3, 4))
```

```
# Output: 24
```

i. ****kwargs:**

Allows a function to accept any number of keyword arguments. It collects all keyword arguments into a dictionary

```
def print_info(**kwargs):  
    for key, value in kwargs.items():  
        print(f"{key}: {value}")
```

```
print_info(name="Alice", age=30,  
city="New York")
```

Break

Time!



Today's Agenda

1



Intro to
Python,
Data Types
& Strings

Break

2



Conditionals
, Functions
in Python

Break

3



NumPy

Break

4



Pandas
Basics



Modules & Packages

Module

- ✓ A module is a file containing Python code. It can define functions, classes, and variables. You can use it to organize your code into reusable pieces.
- ✓ Think of a module as a library in other languages like Java or C#.

```
# mymodule.py
```

```
def greet(name):  
    return f"Hello,  
{name}!"
```

```
def add(a, b):  
    return a + b
```

```
# main.py
```

```
import mymodule
```

```
print(mymodule.greet("Alice"  
) # Output: Hello, Alice!  
print(mymodule.add(3,  
4)) # Output: 7
```

```
python main.py
```

```
Hello, Alice!  
7
```

Package

- ✓ A package is a **collection of modules** organized in directories that includes a special `__init__.py` file.
- ✓ Think of a package as a directory containing multiple related modules or libraries.
- ✓ `__init__.py` is a special Python file used to indicate that a directory is a Python package. It allows the package's modules to be imported together, providing initialization code for the package if needed.
- ✓ `pip` is the package installer for Python. It allows you to install and manage additional libraries and dependencies that are not included in the standard library.

```
mypackage/  
├── __init__.py  
├── module1.py  
└── module2.py
```

```
from mypackage import module1,  
module2  
  
print(module1.function1())  
print(module2.function2())
```

```
python main.py
```

```
Hello, Alice!  
7
```

Virtual Environment

- ✓ A virtual environment is an isolated Python environment that allows you to manage dependencies for your projects independently of each other.
- ✓ To prevent dependency conflicts between projects and to keep the global Python environment clean.
- ✓ Steps: Create, activate, install dependencies, and verify.
- ✓ If you do not use a virtual environment, all the installed packages will be installed globally, potentially causing conflicts with other projects and cluttering your global Python environment.

```
python -m venv projectA_env  
python -m venv projectB_env
```

```
projectA_env\Scripts\activate
```

```
(projectA_env) pip install  
requests==2.24.0
```

Library

- ✓ A library is a collection of **pre-compiled routines** that a program can use. These routines are often packaged into a format that can be easily included and used by various programs, facilitating code reuse and modular programming.
- ✓ Libraries can include various functions, classes, and variables that provide specific functionality. For example, a math library might include routines for mathematical operations, while a graphics library might include routines for rendering images.
- ✓ In Python, a library is typically a package that contains multiple modules. These modules can be related to one another and can be imported into your programs as needed.

```
mathlib/  
├── __init__.py  
├── addition.py  
└── subtraction.py
```

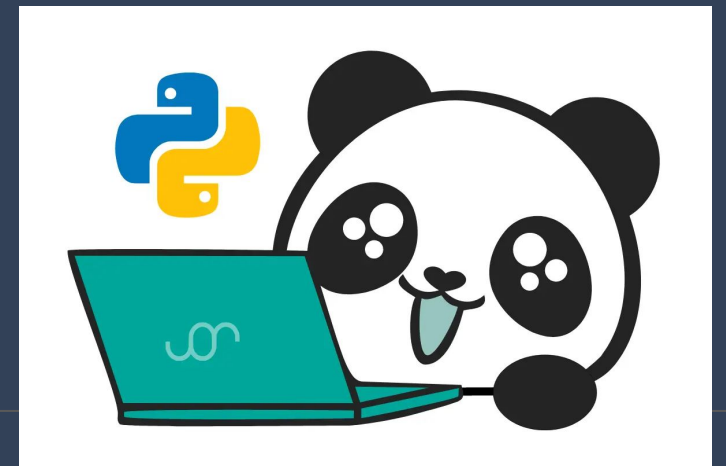
```
from .addition import add  
from .subtraction import subtract  
  
print(add(5, 3))  
print(subtract(5, 3))
```

```
import requests  
response =  
requests.get('https://api.example.  
com/data')  
  
print(response.json())
```

Introduction to Pandas

Introduction

- Pandas is a powerful Python library widely used for *data manipulation* and *analysis*.
- Pandas offers data structures and functions that simplify working with structured data, making it an essential tool in data science and analysis



Why Pandas?

Using Pandas offers several benefits for data manipulation and data analysis:

- Efficient Data Handling
- Data Alignment
- Handling Missing Data
- Data Integration
- Flexible Data Transformation
- Integration with Other Libraries

Data Structures in Pandas

- Pandas **Series** is a one-dimensional array with axis labels.
- Pandas **DataFrame** is two-dimensional data structure with labeled rows and columns.

| Series A | | Series B | | DataFrame | | |
|----------|--------|----------|---------|-----------|--------|---------|
| | apples | | oranges | | apples | oranges |
| 0 | 3 | 0 | 0 | 0 | 3 | 0 |
| 1 | 2 | 1 | 3 | 1 | 2 | 3 |
| 2 | 0 | 2 | 7 | 2 | 0 | 7 |
| 3 | 1 | 3 | 2 | 3 | 1 | 2 |

Filtering a DataFrame - Why do we need filtering?

- When we need to remove redundant or unnecessary data for some tasks
- When we want to filter customers based on certain criteria
- When we need to filter out rows or columns have missing values

Filtering a DataFrame

- Filtering with ``loc`` and ``iloc`` methods
- Filtering by Selecting a Subset of Columns
- Filtering by condition(s)

Filtering a DataFrame - `loc` and `iloc` methods

- **loc** uses row and column labels.
- **iloc** uses row and column indexes.

| | column index | | 0 | 1 | 2 |
|-----------|--------------|---|------|-----|-----------|
| row index | label | | Name | Age | Education |
| 0 | March | 2 | | | |
| 1 | April | 3 | | | |
| 2 | May | 4 | | | |
| 3 | June | 5 | | | |
| 4 | July | 6 | | | |

Filtering a DataFrame - Selecting a Subset of Columns

- Select a single column
- Select multiple columns

| | state | account_length | international_plan | total_intl_calls | churn |
|---|-------|----------------|--------------------|------------------|-------|
| 0 | | | | | |
| 1 | | | | | |
| 2 | | | | | |

Select multiple columns

Select a column

Filtering a DataFrame - Filtering by condition(s)

- Filtering by a condition
- Filtering by multiple conditions

The operators we can use in filtering by condition(s):

- ==: equal
- !=: not equal
- >: greater than
- >=: greater than or equal to
- <: less than
- <=: less than or equal to
- & : and
- | : or



Quiz

Which function should be used to drop a row by index from a DataFrame?

- A** `df.drop()`
- B** `df.delete()`
- C** `df.remove()`
- D** `df.drop_row()`

Quiz

Which function should be used to drop a row by index from a DataFrame?

- A** `df.drop()`
- B** `df.delete()`
- C** `df.remove()`
- D** `df.drop_row()`

Break

Time!



Next Steps

- [Revise the concepts](#) learnt in today's session through class recording and post-class videos
- Attempt the [Assignment & Quiz questions](#) available on Uplevel
- Register for [Technical Coaching Session](#) through Xpert Connect option on Uplevel
- Attend the [Assignment Review Session](#) to get a walkthrough of the assignment solution

What's Next?

To strengthen your knowledge deep dive into NumPy & Pandas.

Refer our post class self paced modules on NumPy & Pandas (available on UpLevel), consume the content at your own pace, practice hard if you stuck somewhere leverage the weekly technical coaching sessions to get your doubts resolved.

Thank
you